Supplementary File for: MaRS-Net: A Transformer-Based Deep Reinforcement Learning Approach for Cooperative Scheduling in Marsupial Robotic Systems

I. MILP Formulation of CMRSP

This section provides a mixed-integer linear programming (MILP) formulation of the Cooperative Marsupial Robot Scheduling Problem (CMRSP). The formulation follows the same carrier–worker workflow, objective, and deterministic travel-time assumptions used in the MDP formulation.

Parameters

$N = \{1, \dots, n\}$	Set of tasks.
$V^+ = \{1, \dots, m^+\}$	Set of CRs.
$V^- = \{1, \dots, m^-\}$	Set of WRs.
$V = V^+ \cup V^-$	Set of all robots.
$\bar{N}$	Node set $\bar{N} = N \cup \{0\}$, where node 0 denotes the start location.
s_i, e_i, p_i, D_i	Source, destination, processing duration, and deadline of task i.
e_0	Common initial location, with $e_0 = (0, 0)$ and $U_0 = C_0 = 0$.
$d(A, B), \tau(A, B)$	Manhattan distance and travel time between locations A and B, where $\tau(A, B) = d(A, B)/v$.
τ^{d}	Docking duration.
τ^{f}	Fetching duration.
τ^{u}	Undocking duration.
ω	Distance–tardiness trade-off weight.
M	A sufficiently large constant.

Decision variables

x_{ij}^r	Binary variable equal to 1 if robot r visits node j immediately after node i, and 0 otherwise.
A_i	Docking start time of task i.
U_i	Undocking time of task i.
C_i	Completion time of task i, i.e., the time when the assigned WR finishes processing.
δ_i	CR travel distance contribution of task i.
ζ_i	Tardiness of task i.

The MILP model for CMRSP is formulated as follows:

$$\min F = \omega \sum_{i \in N} \delta_i + (1 - \omega) \sum_{i \in N} \zeta_i \tag{1}$$

s.t.:

$$\sum_{r \in V^+} \sum_{\substack{i \in \bar{N} \\ i \neq j}} x_{ij}^r = 1, \quad \forall j \in N, \tag{2}$$

$$\sum_{r \in V^+} \sum_{\substack{j \in \bar{N} \\ j \neq i}} x_{ij}^r = 1, \quad \forall i \in N, \tag{3}$$

$$\sum_{r \in V^-} \sum_{\substack{i \in \bar{N} \\ i \neq j}} x_{ij}^r = 1, \quad \forall j \in N, \tag{4}$$

$$\sum_{r \in V^-} \sum_{\substack{j \in \bar{N} \\ j \neq i}} x_{ij}^r = 1, \quad \forall i \in N, \tag{5}$$

$$\sum_{\substack{i \in \bar{N} \\ i \neq j}} x_{ij}^r - \sum_{\substack{i \in \bar{N} \\ i \neq j}} x_{ji}^r = 0, \quad \forall j \in N,\ r \in V, \tag{6}$$

$$\sum_{j \in N} x_{0j}^r \leq 1, \quad \forall r \in V, \tag{7}$$

$$\sum_{i \in N} x_{i0}^r \leq 1, \quad \forall r \in V, \tag{8}$$

$$d(e_j, e_k) + d(e_k, s_i) + d(s_i, e_i) - M(2 - x_{ji}^{r^+} - x_{ki}^{r^-}) \leq \delta_i, \quad \forall i \in N,\ r^+ \in V^+,\ r^- \in V^-,\ j \in \bar{N},\ k \in \bar{N}, \tag{9}$$

$$C_k - M(1 - x_{ki}^{r^-}) \leq A_i, \quad \forall i \in N,\ r^- \in V^-,\ k \in \bar{N}, \tag{10}$$

$$U_j + \tau(e_j, e_k) - M(2 - x_{ji}^{r^+} - x_{ki}^{r^-}) \leq A_i, \quad \forall i \in N,\ r^+ \in V^+,\ r^- \in V^-,\ j \in \bar{N},\ k \in \bar{N}, \tag{11}$$

$$A_i + \tau^{\mathrm{d}} + \tau(e_k, s_i) + \tau^{\mathrm{f}} + \tau(s_i, e_i) + \tau^{\mathrm{u}} - M(1 - x_{ki}^{r^-}) \leq U_i, \quad \forall i \in N,\ r^- \in V^-,\ k \in \bar{N}, \tag{12}$$

$$U_i + p_i \leq C_i, \quad \forall i \in N, \tag{13}$$

$$C_i - D_i \leq \zeta_i, \quad \forall i \in N, \tag{14}$$

$$x_{ii}^r = 0, \quad \forall i \in \bar{N},\ r \in V, \tag{15}$$

$$A_i, U_i, C_i, \delta_i, \zeta_i \geq 0, \quad \forall i \in N, \tag{16}$$

$$x_{ij}^r \in \{0, 1\}, \quad \forall r \in V,\ i, j \in \bar{N}. \tag{17}$$

The objective function (1) minimizes the weighted sum of CR travel distance and task tardiness. Constraints (2)–(5) ensure that each task is visited exactly once by a CR and once by a WR. Constraint (6) enforces route continuity for each robot. Constraints (7) and (8) define the start and end of each robot route at the common node 0, while allowing robots with no assigned tasks to remain unused. Constraint (9) defines the CR travel distance. Constraints (10) and (11) ensure that docking starts only when the assigned WR is available and the assigned CR has reached the WR. Constraint (12) describes the docking, fetching, transfer, and undocking workflow. Constraint (13) defines the completion time after WR processing. Constraint (14) defines tardiness. Constraint (15) eliminates self-visits, and constraints (16) and (17) define the feasible domains of the variables.

II. Wall-Clock Training Time Analysis

Table S.1 reports the per-batch time, per-epoch time, total 100-epoch training time, and peak GPU memory of MaRS-Net, HDRL, and TDRL across six problem scales ($n = 10, 20, 30, 40, 60, 100$). All three methods are measured under the same configuration (batch size 1024, 1,280,000 instances per epoch, and 100 epochs), so the differences mainly reflect architectural factors. Training time increases with n, from roughly 4–5 h at $n = 10$ to 43–60 h at $n = 100$. GPU memory grows more rapidly, reaching approximately 36–39 GB at $n = 100$ because of the $O(n^2)$ self-attention mechanism, and approaches the 48 GB capacity of the modified RTX 4090 GPU used in the experiments.

TABLE S.1: Wall-clock training time and peak GPU memory of three DRL methods across problem scales. All methods use the same training configuration.

n	Method	Batch (s)	Epoch (min)	100-Ep. (h)	GPU (MB)
10	MaRS-Net	0.1274	2.65	4.42	1,540
	HDRL	0.1120	2.33	3.89	1,161
	TDRL	0.1431	2.98	4.97	1,722
20	MaRS-Net	0.2404	5.01	8.35	3,517
	HDRL	0.2167	4.52	7.53	2,854
	TDRL	0.3236	6.74	11.24	3,792
30	MaRS-Net	0.3609	7.52	12.53	6,115
	HDRL	0.3309	6.89	11.49	5,108
	TDRL	0.5142	10.71	17.85	6,498
40	MaRS-Net	0.4776	9.95	16.58	9,050
	HDRL	0.4548	9.47	15.79	7,882
	TDRL	0.6509	13.56	22.60	9,470
60	MaRS-Net	0.7342	15.30	25.49	16,708
	HDRL	0.7277	15.16	25.27	15,028
	TDRL	0.9378	19.54	32.56	17,205
100	MaRS-Net	1.2473	25.99	43.31	38,015
	HDRL	1.4436	30.07	50.12	35,797
	TDRL	1.7232	35.90	59.83	38,776

III. Seed-Sensitivity Analysis

To assess the effect of random seed variability, MaRS-Net is retrained on the 40-task setting under five independent random seeds, with all hyperparameters, data generation rules, optimizer settings, rollout baseline, and decoding protocols kept unchanged. The resulting five checkpoints are evaluated on fixed test sets: Synthetic-20, Synthetic-40, Synthetic-60, and Industrial-40.

Table S.2 reports the mean $\pm$ standard deviation and the coefficient of variation (CV), defined as $\mathrm{CV} = \mathrm{std}/\mathrm{mean} \times 100\%$, across the five independently trained models. Under greedy decoding, the objective CV is 0.65% on Synthetic-40, 0.71% on Synthetic-20, 1.23% on Synthetic-60, and 2.64% on Industrial-40. With Sample-1280 decoding, the objective CV further decreases to 0.34%–1.13%. The distance CV remains below 1% across all test settings, while the tardiness CV is moderately higher due to increased sensitivity to small timing deviations around deadline constraints. The consistently low CV values indicate that random seed variation has limited impact on the performance of MaRS-Net, and the observed results are not driven by a favorable training trajectory. Therefore, the conclusions drawn for MaRS-Net in the main manuscript are robust to seed-induced training variability.

IV. Time–Quality Tradeoff Analysis

To characterize the tradeoff between sampling budget and solution quality, MaRS-Net is evaluated on 100-task instances under varying numbers of sampled trajectories $k \in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024\}$, where $k = 1$ corresponds to greedy decoding and $k > 1$ corresponds to sampling, with the best solution among the k trajectories selected according to the objective function. As shown in Fig. S.1, increasing k yields consistent improvement in solution quality, while the inference time grows gradually and remains within the sub-second range, owing to GPU-parallelized batch decoding. This indicates that MaRS-Net achieves meaningful quality gains with only marginal additional computational cost.

Table S.3 further compares the runtime profile of MaRS-Net with that of ALNS across problem sizes. As an iterative improvement method, ALNS requires a non-negligible initialization phase before producing the first feasible solution. For instance, on 100-task instances, ALNS requires 159.733 ± 18.201 s to obtain an initial feasible solution, while MaRS-Net requires only 0.456 ± 0.056 s (greedy) and 0.580 ± 0.057 s (Sample-1024), corresponding to speedups of approximately $350.6\times$ and $275.6\times$, respectively.

This large runtime gap reflects a fundamental difference in computational paradigms. DRL-based methods such as MaRS-Net adopt a constructive encoder–decoder architecture that generates a complete solution through a single forward pass, bypassing the initialization and iterative refinement phases inherent to metaheuristics. Under the sub-second time scale of DRL inference, metaheuristics cannot produce even an initial feasible solution, rendering a strict matched-budget comparison uninformative in this setting. These results highlight a key advantage of the constructive DRL paradigm for tightly coupled combinatorial optimization. By generating complete solutions in a single forward pass, MaRS-Net maintains sub-second inference across all scales, which is several orders of magnitude faster than iterative metaheuristics. Its solution quality is comparable to that of metaheuristics at small scales, and at medium and large scales where decision coupling intensifies, it significantly outperforms them.

V. Metaheuristic Parameters and Operators

The parameters of the metaheuristic baselines are determined using the Taguchi method for design of experiments under the smaller-the-better criterion for the average objective value. Table S.4 summarizes the final parameter settings and core operators used by each baseline, including the destruction sizes, acceptance-control coefficients, population settings, and neighborhood operators. Full implementations and configurations are provided in the code repository.

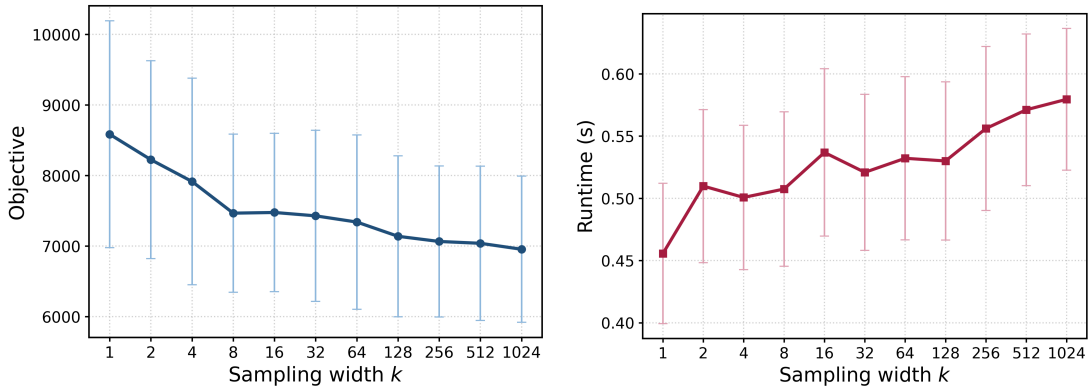
VI. Sensitivity to Reward Trade-Off Parameter ω

To characterize the sensitivity of MaRS-Net to the objective weight parameter ω, separate models are trained with $\omega \in \{0.1, 0.2, \ldots, 0.9\}$ under identical settings. All models are evaluated on 100 randomly generated 40-task instances using both greedy and Sample-1280 decoding. The results are shown in Fig. S.2.

As ω increases, the total travel distance decreases monotonically, while tardiness remains relatively stable for small ω and increases significantly when ω becomes large. The performance varies smoothly with ω, without abrupt changes or sharp sensitivity around any specific value. This indicates that ω serves as a user-defined preference parameter rather than a core tunable component. In the main manuscript, $\omega = 0.4$ is adopted as a practical operating point that balances travel efficiency and service punctuality, and is fixed across all experiments to ensure fair and consistent evaluation.

TABLE S.2: Seed variability of MaRS-Net trained on 40-task instances over five independent seeds.

Decode	Set	Performance across seeds			CV (%)		
		Objective	Distance	Tardiness	Objective	Distance	Tardiness
Greedy	Syn-20	1590.6 ± 11.2	3245.6 ± 24.7	487.2 ± 18.0	0.71	0.76	3.70
	Syn-40	2854.2 ± 18.6	5456.6 ± 43.3	1119.2 ± 29.3	0.65	0.79	2.62
	Syn-60	5235.5 ± 64.3	7677.6 ± 56.5	3607.4 ± 102.6	1.23	0.74	2.84
	Ind-40	3966.9 ± 104.6	5750.2 ± 56.4	2778.1 ± 180.7	2.64	0.98	6.50
Sample1280	Syn-20	1407.4 ± 9.0	3040.9 ± 22.0	318.4 ± 17.0	0.64	0.72	5.35
	Syn-40	2453.9 ± 8.4	5290.8 ± 21.9	562.6 ± 12.7	0.34	0.41	2.25
	Syn-60	4063.7 ± 36.4	7606.3 ± 45.9	1702.0 ± 61.4	0.90	0.60	3.61
	Ind-40	3367.6 ± 37.9	5625.8 ± 24.6	1862.1 ± 64.4	1.13	0.44	3.46



(a) Objective value vs. sampling width k.

(b) Inference time vs. sampling width k.

Fig. S.1: Time–quality tradeoff of MaRS-Net under different sampling widths on 100-task instances.

TABLE S.3: Timing comparison between ALNS and MaRS-Net across problem sizes.

Size	ALNS		MaRS-Net		Ratio	
	Init (s)	Iter (s)	Greedy (s)	Sample-1024 (s)	Init/Greedy	Init/Sample
10	0.122 ± 0.001	0.020 ± 0.000	0.042 ± 0.001	0.062 ± 0.001	2.9×	2.0×
20	0.949 ± 0.016	0.164 ± 0.004	0.093 ± 0.012	0.122 ± 0.000	10.2×	7.8×
40	8.789 ± 0.542	1.539 ± 0.067	0.212 ± 0.000	0.188 ± 0.000	41.4×	46.9×
60	38.973 ± 5.120	10.072 ± 1.847	0.267 ± 0.030	0.317 ± 0.038	145.8×	122.9×
100	159.733 ± 18.201	51.451 ± 12.878	0.456 ± 0.056	0.580 ± 0.057	350.6×	275.6×

TABLE S.4: Main parameters and core operators of the metaheuristic baselines.

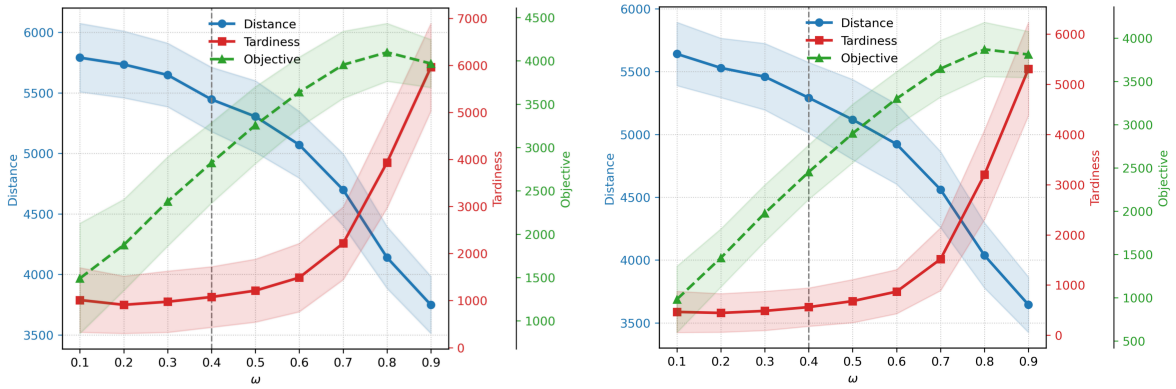
Algorithm	Parameter	Operation
ALNS	$\mu = 10$, $d = \lceil 0.3n \rceil$, $l_s = 15$, $\rho = 0.1$, $\sigma_1 = 33$, $\sigma_2 = 13$, $\sigma_3 = 9$	Random / Worst-Cost Removal; Greedy Insertion
IGA	$\mu = 10$, $d = \lceil 0.3n \rceil$	Referenced Local Search, Random Destruction, Greedy Repair
DABC	$PS = 100$, $PS_e = 50$, $PS_o = 50$, $LIMIT = 1000$, $Rep = 20$	Insertion, Task-Swap Neighborhoods
DIWO	$PS_{init} = 50$, $PS_{max} = 100$, $S_{min} = 1$, $S_{max} = 100$	Insertion, Task-Swap Neighborhoods

n: number of tasks. μ: temperature coefficient in ALNS and IGA. d: destruction size in destroy-repair based search. l_s: operator weight update interval in ALNS. ρ: reaction factor for adaptive operator weight updating in ALNS.
$\sigma_1, \sigma_2, \sigma_3$: scores assigned in ALNS for generating a global-best solution, an improving solution, and an accepted non-improving solution, respectively.
PS: total population size. PS_e: number of employed bees in DABC. PS_o: number of onlooker bees in DABC. $LIMIT$: abandonment threshold in DABC. Rep: repeated onlooker update rounds per DABC iteration.
PS_{init}: initial population size in DIWO. PS_{max}: maximum retained population size in DIWO. S_{min}, S_{max}: minimum and maximum offspring seeds per weed in DIWO.

VII. Gantt Chart and Schedule Compactness Analysis

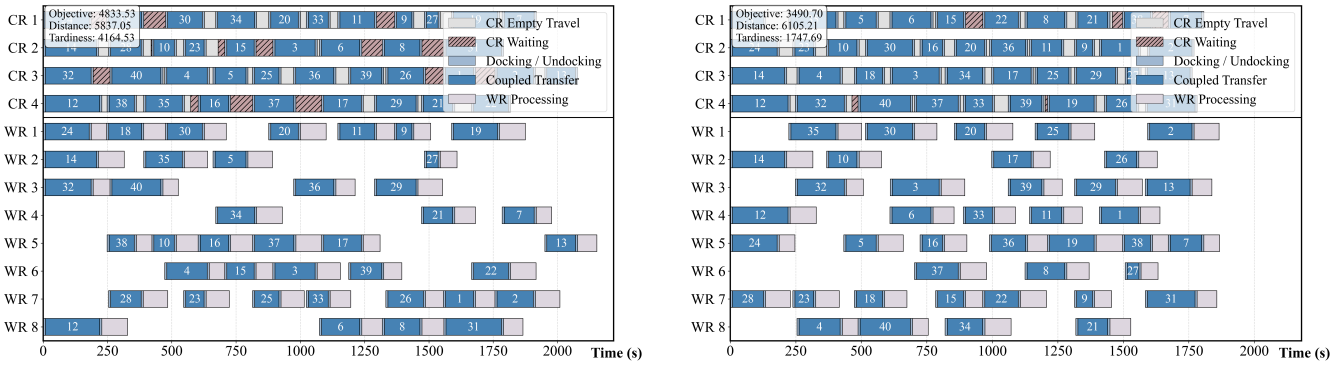
Fig. S.3 presents comparative Gantt charts of the schedules generated by ALNS and MaRS-Net for a 40-task instance. MaRS-Net achieves a substantially lower objective value (3490.70 vs. 4833.53), primarily driven by a significant reduction in tardiness (1747.69 vs. 4164.53), while maintaining comparable travel distance (6105.21 vs. 5837.05).

In terms of schedule compactness, MaRS-Net produces a more tightly coordinated schedule with improved synchronization between carrier and worker operations. Specifically, the CR synchronization waiting time is reduced from 897.84 s under ALNS to 194.30 s under MaRS-Net, corresponding to a reduction of approximately 78.4%. In addition, the total WR



(a) Greedy decoding. (b) Sample-1280 decoding.

Fig. S.2: Sensitivity of MaRS-Net to the trade-off weight ω on 40-task instances.



(a) ALNS (Obj. 4833.53, Dist. 5837.05, Tard. 4164.53). (b) MaRS-Net (Obj. 3490.70, Dist. 6105.21, Tard. 1747.69).

Fig. S.3: Comparative Gantt charts of schedules generated by ALNS and MaRS-Net for a 40-task instance.

idle time decreases from 8051.41 s to 5668.68 s, and the WR utilization rate increases from 53.26% to 62.06%. These results indicate that MaRS-Net not only reduces tardiness but also generates a more compact and better synchronized schedule under carrier-worker coupling constraints.